UNIVERSITY OF CALIFORNIA, MERCED

CSE179: Parallel Computing

March 4

Dong Li

Spring 2026

UCMERCED

www.ucmerced.edu

Today's lecture

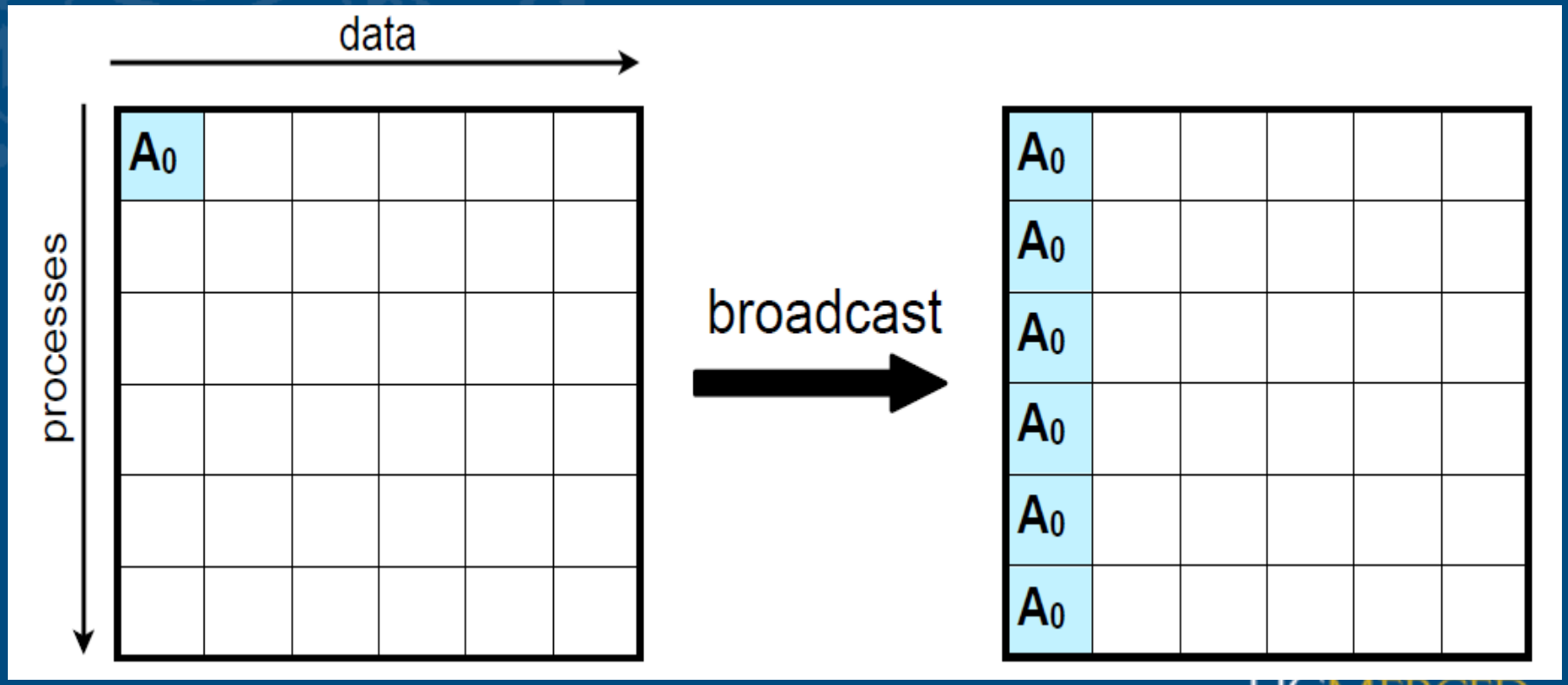
- Collective communications
- MPI topologies
- Data type construction
- MPI threading

Collective Communication in MPI

- MPI provides an extensive set of collective operations
- Operations defined over a communicator's processes
- All processes in a communicator must call the same collective operation
  - e.g. all participants in a one-to-all broadcast call the broadcast primitive, even though all but the root are conceptually just "receivers"
- Simplest collective: barrier synchronization
  - int MPI_Barrier(MPI_Comm comm)
    - Wait until all processes arrive

One-to-all Broadcast

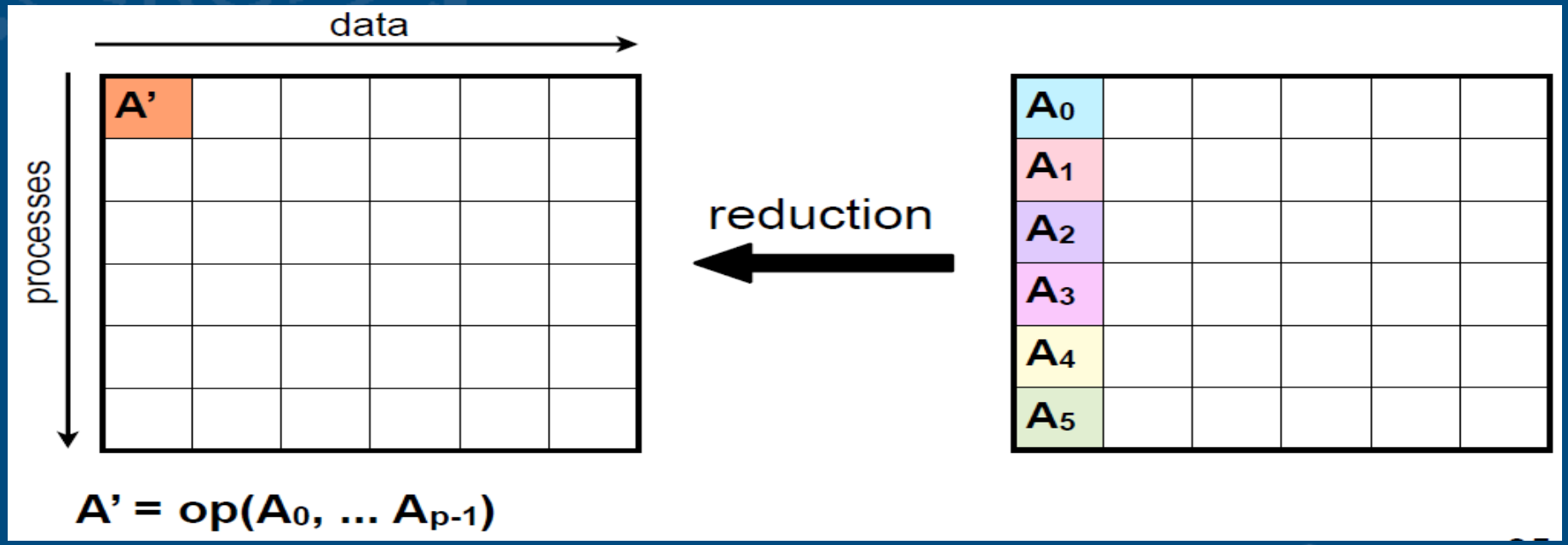
```
int MPI_Bcast(void *buf, int count,
              MPI_Datatype datatype, int source,
              MPI_Comm comm)
```



All-to-one Reduction

```
int MPI_Reduce(void *sendbuf, void *recvbuf,
                    int count, MPI_Datatype datatype,
                    MPI_Op op, int root, MPI_Comm comm)
```

MPI_Op examples: sum, product, min, max, ... (see next page)

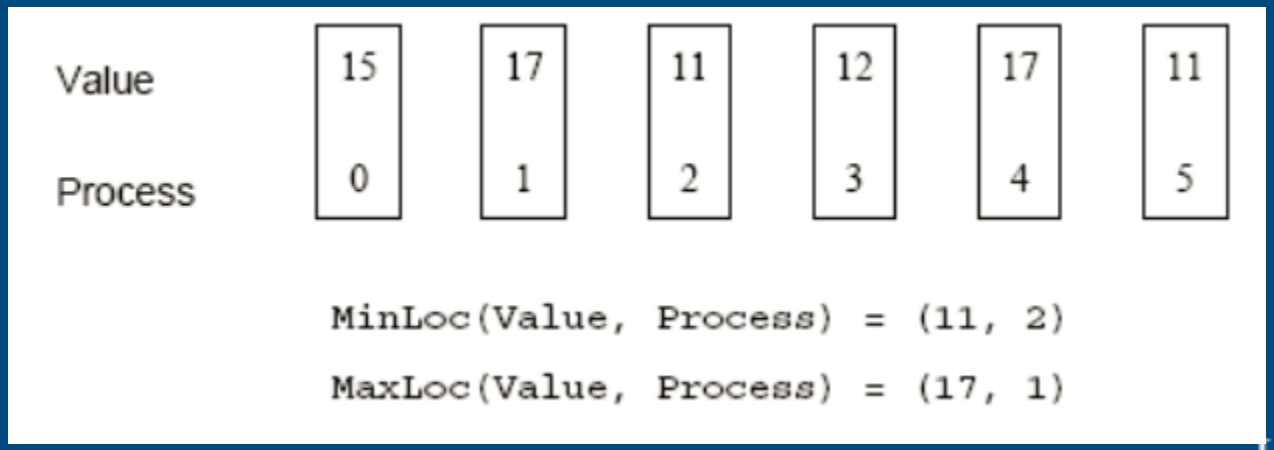


MPI_Op Predefined Reduction Operations

Operation	Meaning	Datatypes
MPI_MAX	Maximum	integers and floating point
MPI_MIN	Minimum	integers and floating point
MPI_SUM	Sum	integers and floating point
MPI_PROD	Product	integers and floating point
MPI_LAND	Logical AND	integers
MPI_BAND	Bit-wise AND	integers and byte
MPI_LOR	Logical OR	integers
MPI_BOR	Bit-wise OR	integers and byte
MPI_LXOR	Logical XOR	integers
MPI_BXOR	Bit-wise XOR	integers and byte
MPI_MAXLOC	Max value-location	Data-pairs
MPI_MINLOC	Min value-location	Data-pairs

MPI_MAXLOC and MPI_MINLOC

- MPI_MAXLOC
  - combines pairs of values (v_i, l_i)
  - returns the pair (v, l) such that
    - v is the maximum among all v_i 's
    - l is the corresponding l_i
      - if non-unique, it is the smallest among l_i 's
- MPI_MINLOC analogous



UCMERCED

Data Types for MINLOC and MAXLOC Reductions

- MPI_MAXLOC and MPI_MINLOC reductions operate on data pairs

MPI Datatype	C Datatype
MPI_2INT	**pair of** ints
MPI_SHORT_INT	short **and** int
MPI_LONG_INT	long **and** int
MPI_LONG_DOUBLE_INT	long double **and** int
MPI_FLOAT_INT	float **and** int
MPI_DOUBLE_INT	double **and** int

All-to-All Reduction and Prefix Sum

- All-to-all reduction - every process gets a copy of the result

```
int MPI_Allreduce(void *sendbuf, void *recvbuf,
                  int count, MPI_Datatype datatype,
                  MPI_Op op, MPI_Comm comm)
```

  - semantically equivalent to MPI_Reduce + MPI_Bcast
- Parallel prefix operations
  - inclusive scan: processor *i* result = *op*(*v0*, ... *vi*)

```
int MPI_Scan(void *sendbuf, void *recvbuf,
             int count, MPI_Datatype datatype, MPI_Op op,
             MPI_Comm comm)
```

  - exclusive scan: processor *i* result = *op*(*v0*, ... *vi*-1)

```
int MPI_Exscan(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op,
               MPI_Comm comm)
```

Exscan example MPI_SUM	input	[2	4	1	1	0	1	-3	2	0	6	1	5]
	output	[0	2	6	7	8	8	9	6	8	8	14	15]

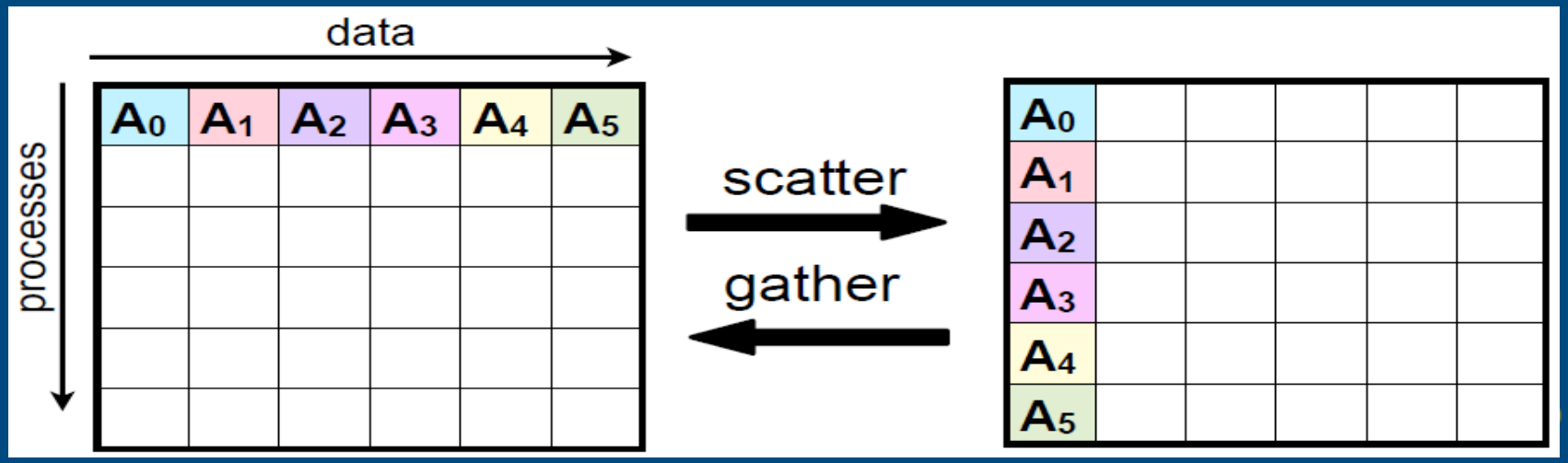
Scatter/Gather

- Scatter data p-1 blocks from root process delivering one to each other

```
int MPI_Scatter(void *sendbuf, int sendcount,
                MPI_Datatype senddatatype, void *recvbuf,
                int recvcount, MPI_Datatype recvdatatype,
                int source, MPI_Comm comm)
```

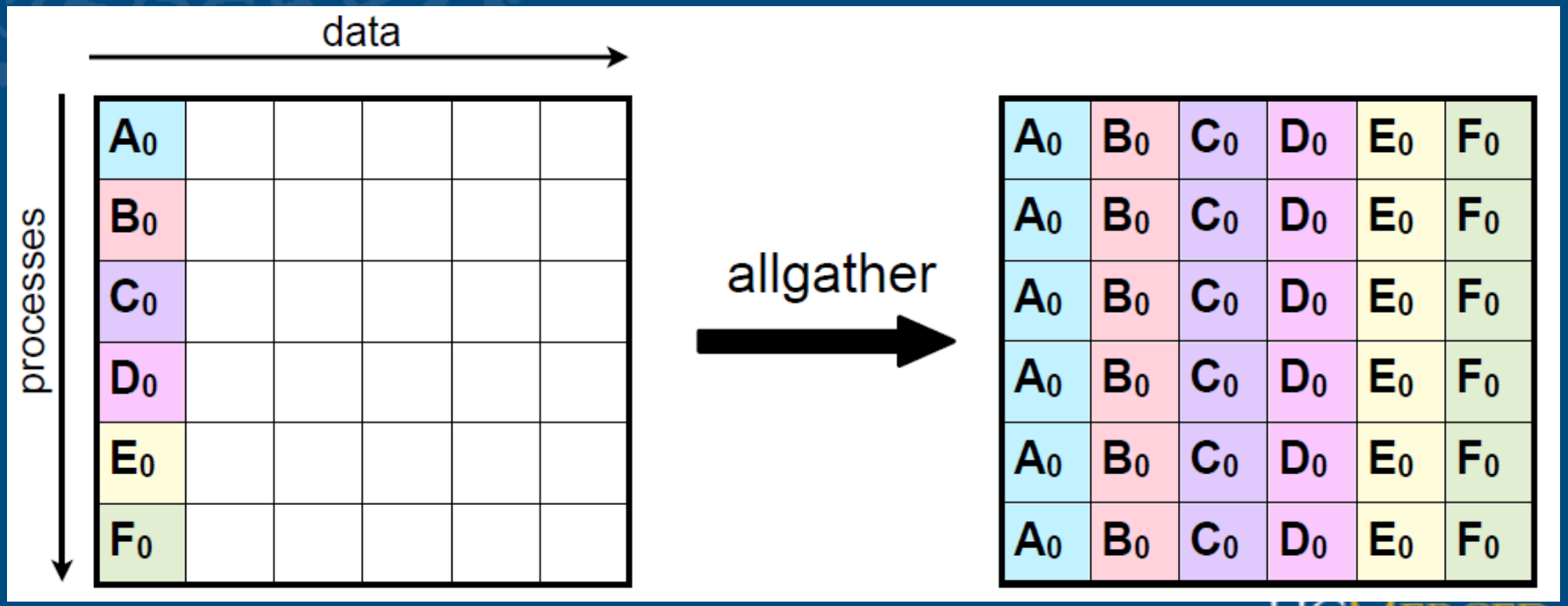
- Gather data at one process

```
int MPI_Gather(void *sendbuf, int sendcount,
               MPI_Datatype senddatatype, void *recvbuf,
               int recvcount, MPI_Datatype recvdatatype,
               int target, MPI_Comm comm)
```



Allgather

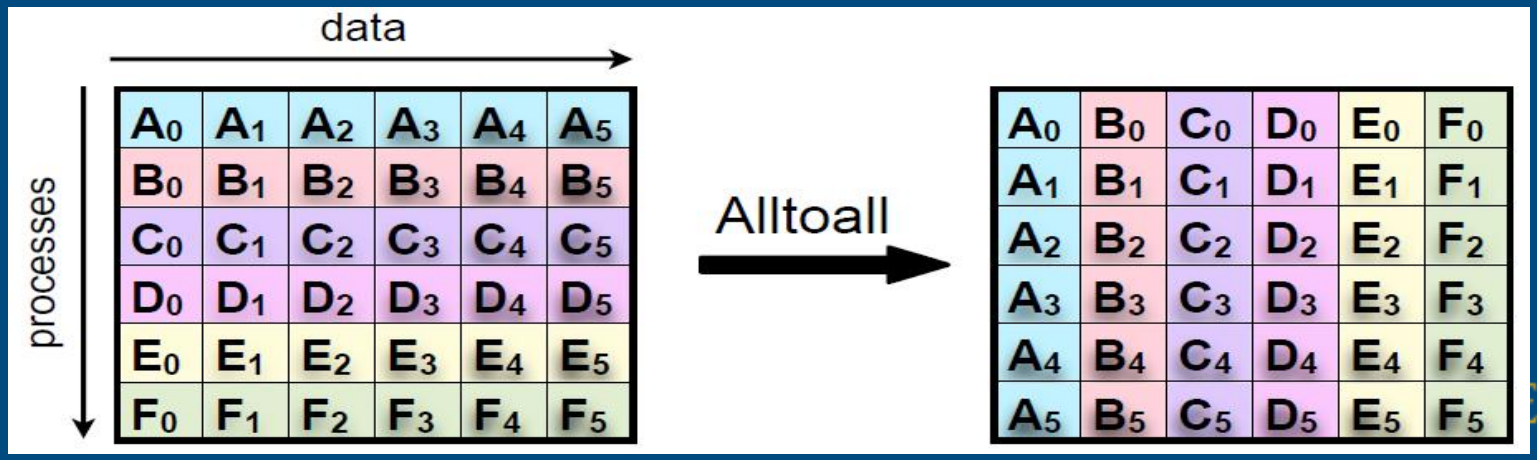
```
int MPI_AllGather(void *sendbuf, int sendcount,
                  MPI_Datatype senddatatype, void *recvbuf,
                  int recvcount, MPI_Datatype recvdatatype,
                  MPI_Comm comm)
```



All-to-All Personalized Communication

- Each process starts with its own set of blocks, one destined for each process
- Each process finishes with all blocks destined for itself
- Analogous to a matrix transpose

```
int MPI_Alltoall(void *sendbuf, int sendcount,
                 MPI_Datatype senddatatype, void *recvbuf,
                 int recvcount, MPI_Datatype recvdatatype,
                 MPI_Comm comm)
```



Splitting Communicators

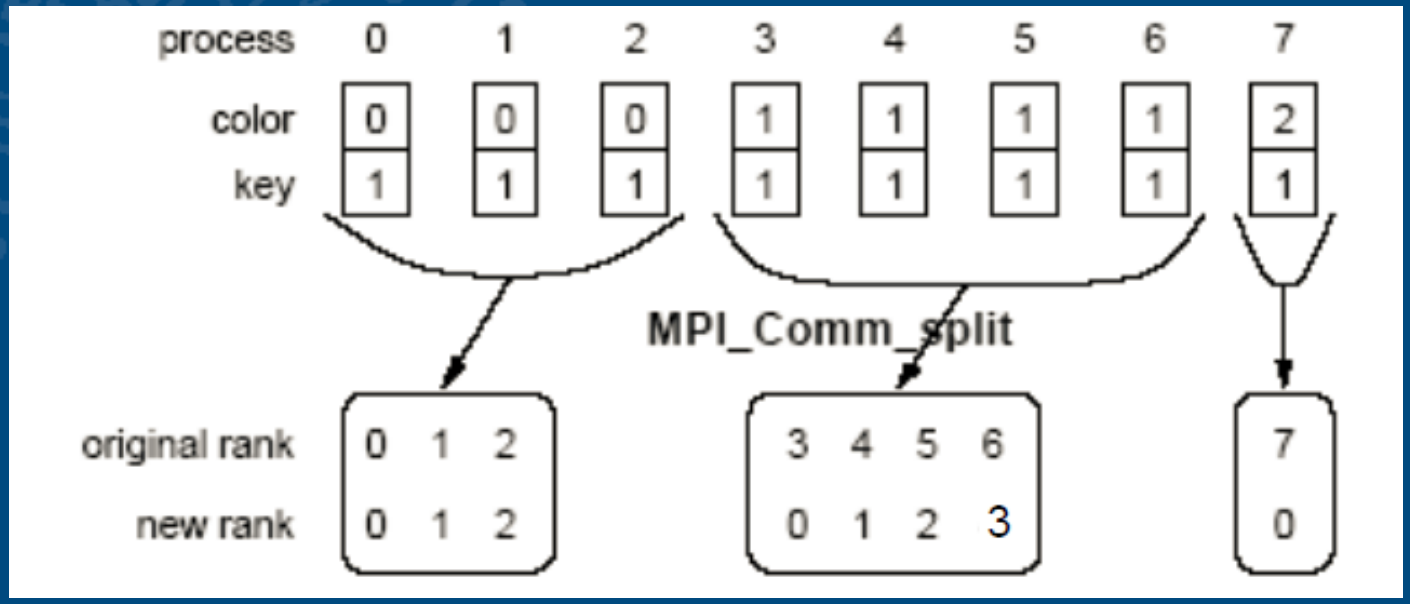
- Useful to partition communication among process subsets
- MPI provides mechanism for partitioning a process group
  - splitting communicators
- Simplest such mechanism

```
int MPI_Comm_split(MPI_Comm comm, int color, int key,
                   MPI_Comm *newcomm)
```

  - Effects
    - group processes by color
    - sort resulting groups by key

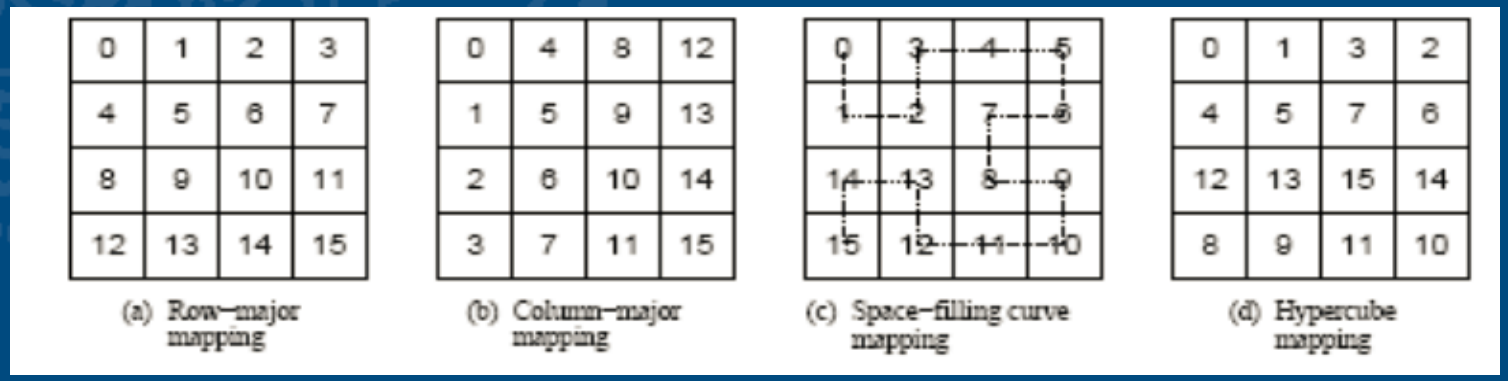
Splitting Communicators

- Using MPI_Comm_split to split a group of processes in a communicator into subgroups



Topologies and Embeddings

- Processor ids in MPI_COMM_WORLD can be remapped
  - higher dimensional meshes
  - space-filling curves



- Goodness of any mapping
  - determined by the interaction pattern (program, topology of the machine)
  - MPI does not provide any explicit control over these mappings

UCMERCED

Cartesian Topologies

- For regular problems a multidimensional mesh organization of processes can be convenient
- Creating a new communicator augmented with a mesh view

```
int MPI_Cart_create(MPI_Comm comm_old, int ndims,
                    int *dims, int *periods, int reorder,
                    MPI_Comm *comm_cart)
```

- Map processes into a mesh
  - ndims = number of dimensions
  - dims = vector with length of each dimension
  - periods = vector indicates which dims are periodic
  - reorder = flag - ranking may be reordered
- Processor coordinate in cartesian topology
  - a vector of length ndims

Using Cartesian Topologies

- Sending and receiving still requires 1-D ranks
- Map Cartesian coordinates ⇔ rank

```
int MPI_Cart_coord(MPI_Comm comm_cart, int rank, int
                   maxdims, int *coords)
int MPI_Cart_rank(MPI_Comm comm_cart, int *coords, int
                  *rank)
```

- Most common operation on cartesian topologies is a shift
- Determine the rank of source and destination of a shift

```
int MPI_Cart_shift(MPI_Comm comm_cart, int dir, int s_step,
                   int *rank_source, int *rank_dest)
```

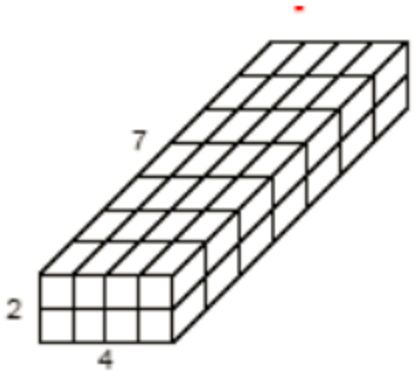
Splitting Cartesian Topologies

- Processes arranged in a virtual grid using Cartesian topology
- May need to restrict communication to a subset of the grid
- Partition a Cartesian topology to form lower-dimensional grids

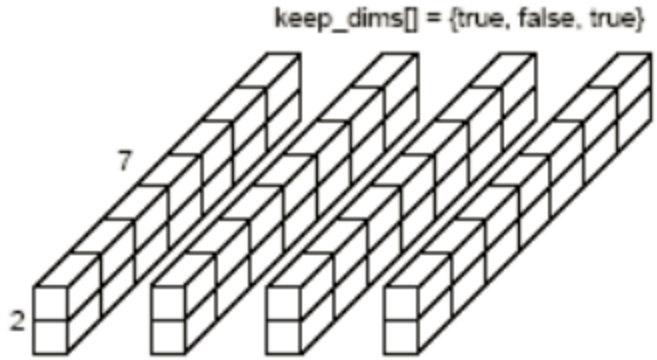
```
int MPI_Cart_sub(MPI_Comm comm_cart, int *keep_dims,
                 MPI_Comm *comm_subcart)
```

- If keep_dims[i] is true (i.e. non-zero in C)
  - *i*th dimension is retained in the new sub-topology
- Process coordinates in a sub-topology
  - derived from coordinate in the original topology
  - disregard coordinates for dimensions that were dropped

Splitting Cartesian Topologies

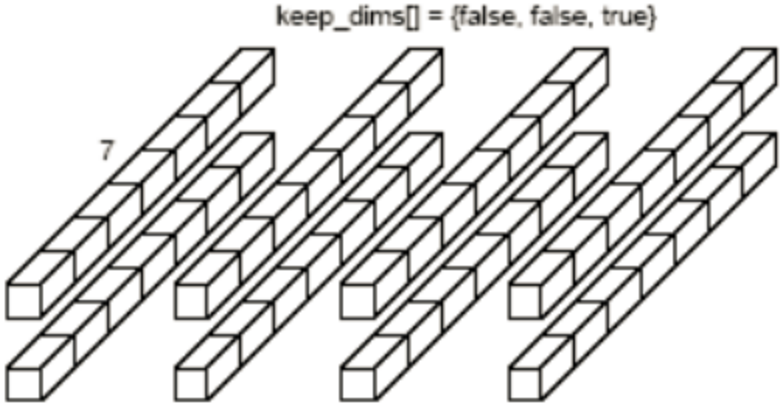


2 x 4 x 7



(a)

4 @ 2 x 1 x 7



(b)

8 @ 1 x 1 x 7

Graph Topologies

- For irregular problems a graph organization of processes can be convenient

```
int MPI_Graph_create(MPI_Comm comm_old, int nnodes,
                     int *index, int *edges,
                     int reorder, MPI_Comm *cgraph)
```

- Map processes into a graph
  - nnodes = number of nodes
  - index = vector of integers describing the total number of neighbors of the first i graph nodes
  - edges = vector of integers describing edges
  - reorder = flag indicating ranking may be reordered

process	neighbors
0	1, 3
1	0
2	3
3	0, 2

nnodes = 4
index = 2, 3, 4, 6
edges = 1, 3, 0, 3, 0, 2

Operations on Graph Topologies

- Interrogating a graph topology with MPI_Graphdims_get

```
int MPI_Graphdims_get(MPI_Comm comm, int *nnodes, int
                      *nedges)
```

- Extracting a graph topology with MPI_Graph_get
  - read out the adjacency list structure in index and edges

```
int MPI_Graph_get(MPI_Comm comm, int maxindex,
                  int maxedges, int *index,
                  int *edges)
```

MPI Derived Data Types

- A general datatype is an opaque object that specifies 2 things
  - a sequence of basic data types
  - a sequence of integer (byte) displacements

- Type *map* = pair of type & displacement sequences (equivalently, a sequence of pairs)

```
int MPI_Type_struct(int count, int blocklens[],
                    MPI_Aint indices[], MPI_Datatype old_types[],
                    MPI_Datatype *newtype )
```

Building an MPI Data Type

```
int MPI_Type_struct(int count, int blocklens[],
                    MPI_Aint indices[], MPI_Datatype old_types[],
                    MPI_Datatype *newtype )
```

Example

```
struct { int a; char b; } foo;

blen[0]=1; indices[0] = 0;                oldtypes[0]=MPI_INT;
blen[1]=1; indices[1] = &foo.b - &foo;    oldtypes[1]=MPI_CHAR;
blen[2]=1; indices[2] = sizeof(foo);      oldtypes[2]= MPI_UB;
MPI_Type_struct( 3, blen, indices, oldtypes, &newtype );
```

MPI Data Type Constructor Example 1

int MPI_Type_contiguous(int count, MPI_Datatype oldtype,
MPI_Datatype *newtype)

- newtype is the datatype obtained by concatenating count copies of oldtype

Example

consider constructing newtype from the following

- oldtype with type map *{ (double, 0), (char, 8) }* with extent 16
- Let count = 3

type *map* of newtype is

- *{ (double, 0), (char, 8),*
- *(double, 16), (char, 24),*
- *(double, 32), (char, 40) }*

namely, alternating double and char elements, with

- displacements *0, 8, 16, 24, 32, 40*

UCMERCED

MPI Data Type Constructor Example 2

```
int MPI_Type_vector(int count, int blocklength, int stride,
                    MPI_Datatype oldtype, MPI_Datatype *newtype)
```

- Let oldtype have type *map*
  - { (double, 0), (char, 8) } with extent 16
- A call to MPI_Type_vector(2, 3, 4, oldtype, newtype) will create the datatype with type *map*
  - two blocks with three copies each of the old type, with a stride of 4 elements (*4* x *16* bytes) between the blocks

{ (double, 0), (char, 8), (double, 16), (char, 24), (double, 32), (char, 40), (double, 64), (char, 72), (double, 80), (char, 88), (double, 96), (char, 104) }

Threads and MPI

- MPI does not define if an MPI process is a thread or an OS process
  - threads are not addressable
  - MPI_Send(... thread_id ...) is not possible
- MPI-2 Specification
  - does not mandate thread support
  - specifies what a thread-compliant MPI should do
  - specifies four levels of thread support

Initializing MPI for Threading

int MPI_Init_thread(int *argc, char ***argv, int *required*, int *provided)

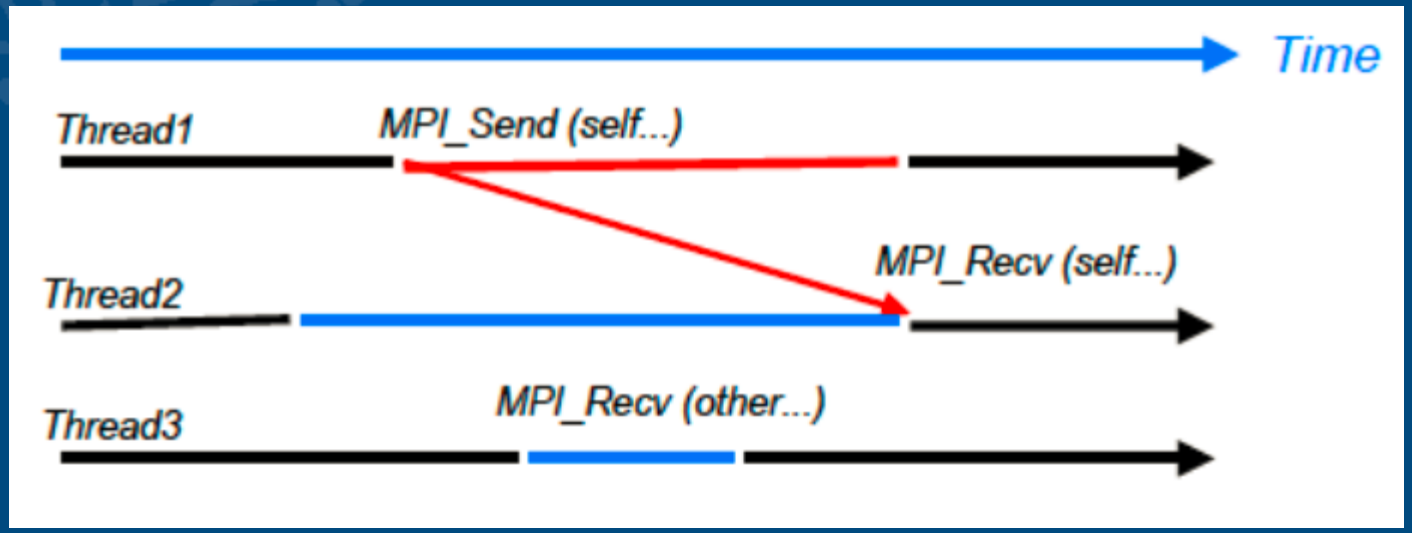
- Used instead of MPI_Init; MPI_Init_thread has a provision to request a certain level of thread support in *required*
  - MPI_THREAD_SINGLE: only one thread will execute
  - MPI_THREAD_FUNNELED: if the process is multithreaded, only the thread that called MPI_Init_thread will make MPI calls
  - MPI_THREAD_SERIALIZED: if the process is multithreaded, only one thread will make MPI library calls at one time
  - MPI_THREAD_MULTIPLE: if the process is multithreaded, multiple threads may call MPI at once with no restrictions

- Require the lowest level that you need
- MPI_Init is equivalent to supplying MPI_THREAD_SINGLE to MPI_Init_thread

UCMERCED

Thread-compliant MPI

- All MPI library calls are thread safe
- Blocking calls block the calling thread only
  - other threads can continue executing



MPI Threading Inquiry Primitives

- Inquire about what kind of thread support MPI has provided to your application

```
int MPI_Query_thread(int *provided)
```

- Inquire whether this thread called MPI_Init or MPI_Init_thread

```
int MPI_Is_thread_main(int *flag)
```

MPI + Threading Example

```
#include "mpi.h"
#include <stdio.h>

int main( int argc, char *argv[] )
{
   int errs = 0;
   int provided, flag, claimed;
   pthread_t thread;

   MPI_Init_thread( 0, 0, MPI_THREAD_MULTIPLE, &provided );

   MPI_Is_thread_main( &flag );
   if (!flag) {
      errs++;
      printf( "This thread called init_thread but Is_thread_main gave false\n" );
      fflush(stdout);
   }
   MPI_Query_thread( &claimed );
   if (claimed != provided) {
      errs++;
      printf( "Query thread gave thread level %d but Init_thread gave %d\n", claimed, provided );
      fflush(stdout);
   }
   pthread_create(&thread, NULL, mythread_function, NULL);
   ...

   MPI_Finalize();
   return errs;
}
```

One-Sided vs. Two-Sided Communication

- Two-sided: data transfer and synchronization are conjoined
  - message passing communication is two-sided
    - sender and receiver issue explicit send or receive operations to engage in a communication
- One-sided: data transfer and synchronization are separate
  - a process or thread of control can read or modify remote data without explicit pairing with another process
  - Terms
    - origin process: process performing remote memory access
    - target process: process whose data is being accessed

Why One-Sided Communication?

- If communication pattern is not known a priori, using a two-sided (send/recv) model requires an extra step to determine how many sends-recvs to issue on each processor
- Easier to code using one-sided communication because only the origin or target process needs to issue the put or get call
- Expose hardware shared memory
  - more direct mapping of communication onto HW using load/store
    - avoid SW overhead of message passing; let the HW do its thing!

One-Sided Communication in MPI-2

- MPI-2 Remote Memory Access (RMA)
  - processes in a communicator can read, write, and accumulate values in a region of “shared” memory
- Two aspects of RMA-based communication
  - data transfer, synchronization
- RMA advantages
  - multiple data transfers with a single synchronization operation
  - can be significantly faster than send/recv on some platforms
    - e.g. systems with hardware support for shared memory

MPI-2 RMA Operation Overview

- MPI_Win_create
  - collective operation to create new window object
  - exposes memory to RMA by other processes in a communicator
- MPI_Win_free
  - deallocates window object
- Non-blocking data movement operations
  - MPI_Put
    - moves data from local memory to remote memory
  - MPI_Get
    - retrieves data from remote memory into local memory
  - MPI_Accumulate
    - updates remote memory using local values
- Synchronization operations

Active Target vs. Passive Target RMA

- Passive target RMA
  - target process makes no synchronization call
- Active target RMA
  - requires participation from the target process in the form of synchronization calls (fence or post/wait, start/complete)

Synchronization for Passive Target RMA

- MPI_Win_lock(locktype, target_rank, assert, win) "beginning RMA"
  - locktype values
    - MPI_LOCK_EXCLUSIVE
      one process at a time may access
      use when modifying the window
    - MPI_LOCK_SHARED
      multiple processes
      (as long as none hold MPI_LOCK_EXCLUSIVE)
      useful when accessing window only with MPI_Get
  - assert values
    - 0 or MPI_MODE_NOCHECK
- MPI_Win_unlock(target_rank, win) "ending RMA"

Active Target Synchronization

- MPI_Win_start
  - begins an RMA epoch on origin process
- MPI_Win_post
  - starts RMA exposure for a local window on a target process
- MPI_Win_wait/test
  - end RMA exposure on local window on a target process
- MPI_Win_complete
  - forces local completion an RMA epoch on origin
- MPI_Win_fence
  - collective forces remote completion of put/get/acc before fence

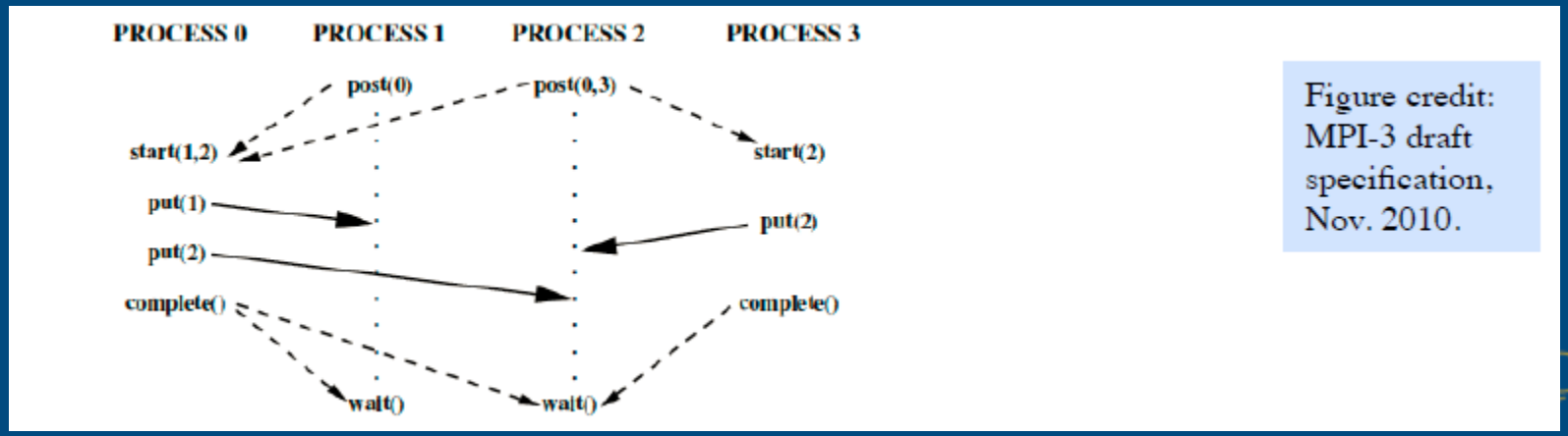


Figure credit: MPI-3 draft specification, Nov. 2010.

MPI RMA Active Target Example

Generic loosely synchronous, iterative code

The window at each process consists of array A, which contains the origin and target buffers of the Get calls

```
...
while(!converged(A)){
   update_boundary(A);
   MPI_Win_post(togroup, win);
   MPI_Win_start(fromgroup, win);
   for(i=0; i < toneighbors; i++)
      MPI_Get(&tobuf[i], 1, totype[i], fromneighbor[i],
              fromdisp[i], 1, fromtype[i], win);
   update_core(A);
   MPI_Win_complete(win);
 }
```

Similar code could be written with Put rather than Get

UCMERCED

MPI-1 Profiling Interface - PMPI

- To support tools, MPI implementations define two interfaces to every MPI function
  - MPI_xxx
  - PMPI_xxx
- One can "wrap" MPI functions with a tool library to observe execution of an MPI program

```
int MPI_Send(void* buffer, int count, MPI_Datatype dtype,
             int dest, int tag, MPI_Comm comm)
{
  double tstart = MPI_Wtime(); /* Pass on all arguments */
  int extent;
  int result = PMPI_Send(buffer,count,dtype,dest,tag,comm);
  MPI_Type_size(datatype, &extent); /* Compute size */
  totalBytes += count*extent;
  totalTime += MPI_Wtime() - tstart; /* and time */
  return result;
}
```

Some MPI Tools

- MPICH MPI implementation
  - MPI tracing library
  - Jumpshot trace visualization tool
- OpenMPI MPI implementation
- Vampir: MPI trace analysis tools
  - http://www.vampir.eu/
- mpiP library for profiling MPI operations
  - http://mpip.sourceforge.net
- Memcheck
  - OpenMPI + valgrind checks correct use of comm buffers
  - http://www.open-mpi.org
- Marmot
  - checks usage of MPI routines
  - http://www.hlrs.de/organization/av/amt/projects/marmot

UCMERCED

MPI Libraries

- SCALAPACK - dense linear algebra using block-cyclic tilings
  - http://www.netlib.org/scalapack/scalapack_home.html
- PetSC - Portable Extensible, Toolkit for Scientific Computation
  - data structures and routines for solution of scientific applications modeled by partial differential equations
  - http://www.mcs.anl.gov/petsc/petsc-as
- Trilinos - software framework for solving large-scale, complex multi-physics engineering and scientific problems
  - http://trilinos.sandia.gov

MPI-3 Additions

- Nonblocking collective operations
  - barrier synchronization
  - Broadcast
  - Gather
  - Scatter
  - gather-to-all
  - all-to-all scatter/gather
  - Reduce
  - reduce-scatter
  - inclusive scan
  - exclusive scan

Building MPI Programs

- Each MPI installation defines compilation scripts
  - mpicc: C
  - mpif90: Fortran 90
  - mpif77: Fortran 77
  - mpicxx, mpiCC: C++
- Benefits of using these scripts
  - they supply the appropriate paths
    - for MPI include files
    - for MPI library files
  - they link appropriate libraries into your executable

Common Errors and Misunderstandings

- Expecting argc and argv to be passed to all processes
  - some MPI implementations pass them to all processes, but the MPI standard does not require it
- Doing things before MPI_Init or after MPI_Finalize
  - the MPI standard says nothing about the state of an execution outside this interval
- Matching MPI_Bcast with MPI_Recv; all should use MPI_Bcast
- Assuming your MPI implementation is thread safe

Running MPI Programs

- Each MPI installation provides one or more launch scripts
  - mpirun
  - mpiexec

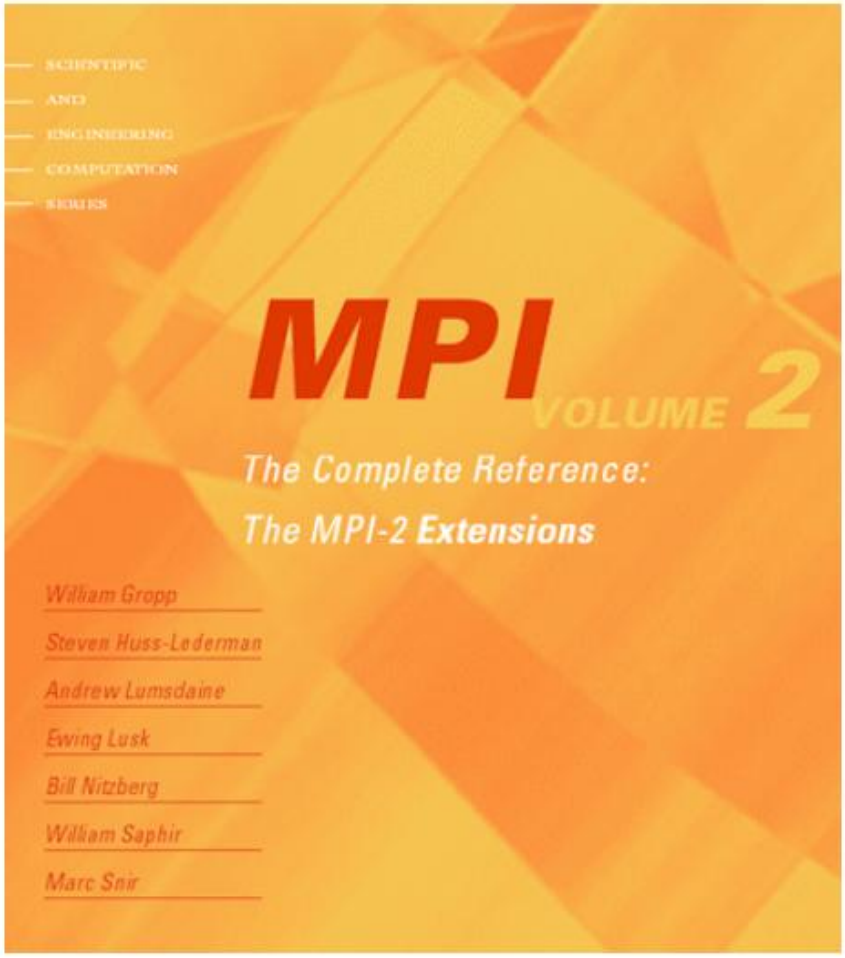
- On networks of workstations, launch MPI as follows
  - mpirun [-np PE] [--hostfile <filename>] <pgm>
    - mpirun will use rsh or ssh to launch jobs on machines in hostfile
    - without a hostfile, it will run all jobs on the local node

MPI Online Resources

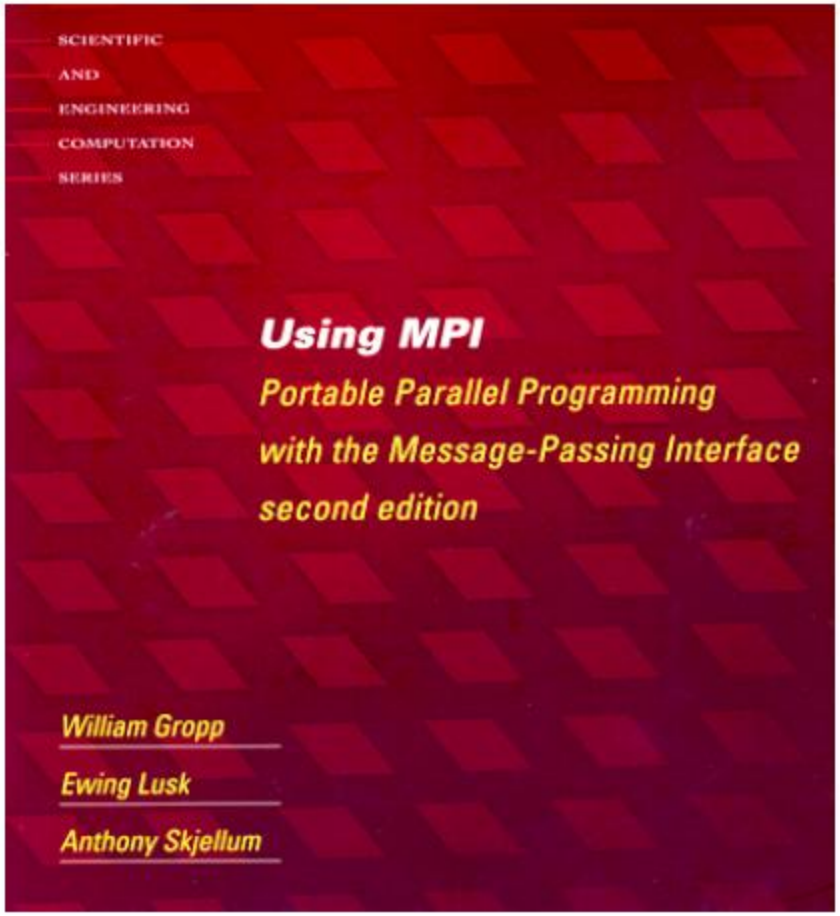
- http://www.mpi-forum.org
  - http://www.mpi-forum.org/docs/docs.html
    - MPI standards documents (all official releases)

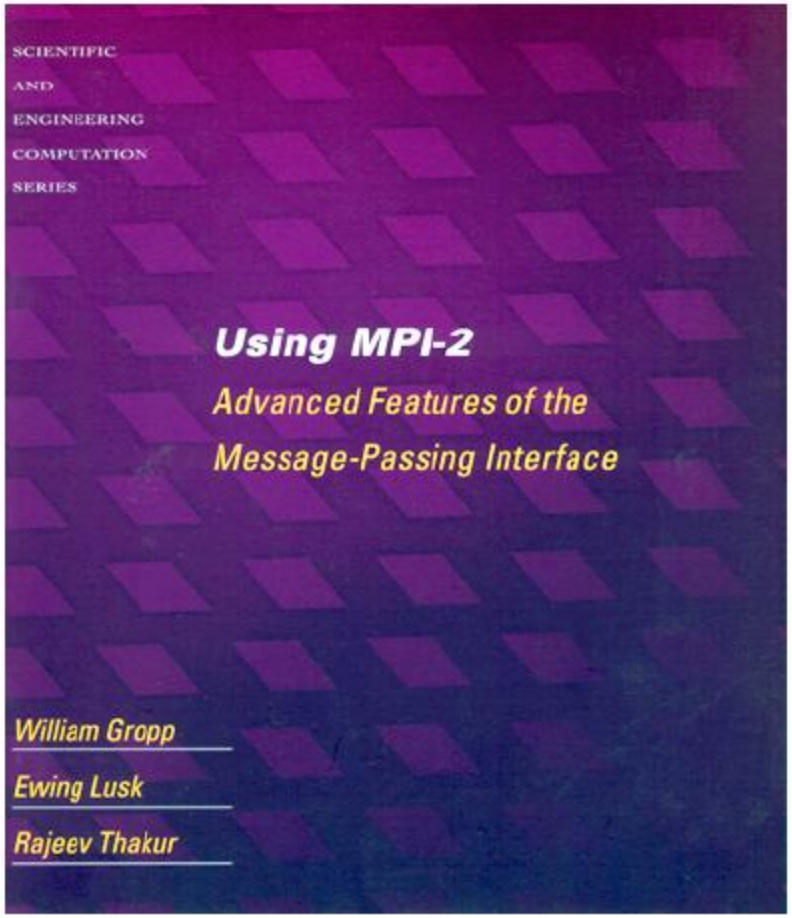
- http://www.mcs.anl.gov/research/projects/mpi/
  - tutorials http://www.mcs.anl.gov/research/projects/mpi/learning.html

The MPI and MPI-2 Standards



Guides to MPI and MPI-2 Programming





Reading Materials

- Chapter 6
  - 6.4-6.7

UCMERCED

Slides references

- Sample slides accompanied with the book “parallel computing”
- William Gropp, Ewing Lusk and Anthony Skjellum. Using MPI, 2nd Edition Portable Parallel Programming with the Message Passing Interface. MIT Press, 1999; ISBN 0-262-57132-3
- John Mellor-Crummey (Rice), COMP422 slides from Spring 2014